DECODE – SRS Presentation Speech (simple English)

Cover — Sude

Good morning. We are the DECODE team: Burcu, İrem, Mehmet and me, Sude. DECODE means Developer Experience Classification via Eye-tracking Data Evaluation. Our supervisor is Yeliz Yeşilada, and our co-supervisor is Şükrü Eraslan. Today we will show you the main parts of our SRS.

Slide 1 · Problem, aim, scope — Sude

In a remote interview, the interviewer sees the candidate's answer. But they cannot see how the candidate reads the code. Studies show that novices and experienced programmers read the same code in a different order.

Our aim is to find out if a candidate is a novice or an experienced programmer. We do this by looking at the order of their eye movements on the code. We only use a normal webcam.

Here is our scope. We change an open-source video call tool, MiroTalk, so it records the candidate's webcam. We find where the candidate looks on the code and build their scanpath. Then we compare it with two reference paths, one for novices and one for experienced programmers. We build these paths with STA, from public datasets.

We always ask for consent first, and we delete the video after the analysis. Our result only helps the interviewer. It does not make the hiring decision.

Other studies used STA to describe groups of programmers. Other studies also used eye tracking to find novices, but with machine learning. As far as we know, we are the first to use STA to classify a single programmer.

Slide 2 · Related work — Sude

In our SRS we compare eleven studies. This table shows the most important ones.

Some studies show that experience changes how people look at code, but they only compare groups. Two studies classify single people with machine learning. They reach 72 and 76 percent. But they only count fixations and durations. They do not use the reading order.

Eraslan and colleagues used STA to detect autism. They reached about 60 percent. Another study used STA on code, but only to describe groups, not to classify people.

Fang and colleagues show that a webcam can follow reading on code, but not at word level.

The last row shows the gap. No study does all four things: classify a person, use reading order, work on code and use a webcam. DECODE does all four.

Why do we use STA? It has four main benefits. It uses the reading order. Its result is easy to

explain. It does not need training, so small datasets are enough. And it is fast.

It also has risks. The earlier STA classifier reached only about 60 percent. One path for a whole group can hide differences between people. We can only use the code snippets in the datasets, and only Java. And the datasets come from lab eye trackers, but our candidates use webcams.

Nobody has tested if reading order works better than the other features. We will test it.

Slide 3 · Context diagram — Mehmet, then Burcu

Mehmet: Thank you, Sude. This is our context diagram. DECODE works with two outside systems.

On the left is MiroTalk. First we looked at Google Meet. But Google Meet does not give access to new projects, and it lowers the video quality when the internet is slow. So we chose MiroTalk. It is open source and it runs in the browser, so there is nothing to install. We record the candidate's webcam on their own computer, at 720p or higher. After the call, we upload the video.

Burcu: On the right is the public eye-tracking dataset. We use it to build the reference paths.

We researched three topics at the same time: the video call tool, the datasets, and how well the data fits STA. Sude and I looked at ten public datasets. We used three rules. The dataset must use one programming language. It must have novice and experienced labels. And everyone must read the same code.

Three datasets passed. EMIP is the biggest, with 216 people, but it has only two programs. Corrected EMIP is a cleaned version with 159 people. Peitek has fewer people, 31, but it has ten programs, and the code blocks are already marked.

So we start with Peitek. When our system works on Peitek, we will test it on EMIP and corrected EMIP too, and compare the results.

Slide 4 · Key user interfaces — İrem

Thank you, Burcu. DECODE is a side panel inside the MiroTalk call. These are the four main screens.

The first screen is consent. The candidate accepts or declines before any recording starts.

The second screen is calibration. The candidate looks at nine points on the screen, so the system learns their eye position.

The third screen is the live session. The interviewer follows the recording while the candidate reads the code.

The fourth screen is the report. After the session, the interviewer sees the result for each candidate. It shows novice or experienced, the confidence, and how reliable the result is.

The report is simple, so you do not need eye-tracking knowledge to read it.

Slide 5 · Critical requirements — İrem

DECODE has nine main functions.

During the call, we set up the session, ask for consent, calibrate and record. After the call, we map the eye movements to the code, build the scanpath, classify the candidate and show the report. Function nine stores the reference paths.

Three functions are the most critical.

The first is consent, F2. Face and eye data are personal data under GDPR and KVKK. Without consent, we record nothing.

The second is mapping to code, F5. A webcam is not very accurate, so we cannot find the exact line. That is why we use code blocks.

The third is classification, F7. We compare the candidate's path with the novice path and the experienced path. The candidate goes to the group that is more similar.

Slide 6 · Use cases — İrem

Our use case diagram has three actors: the interviewer, the candidate and MiroTalk. It has eight use cases, and two of them are critical.

The first is Run Analysis Session. The interviewer starts it. The candidate gives consent and does the calibration. Then the candidate reads the code while the webcam records. When it ends, the system uploads the video and finds the eye movements on the code. It compares them with the two reference paths and saves the result. Then it deletes the video.

There are two other cases. If the candidate says no, we record nothing. If the system loses the face, it warns the interviewer and starts a new calibration.

The second is View Classification Report. The interviewer chooses a session and sees each candidate's result. If the calibration error is too high, the result is marked as not reliable.

Slide 7 · Usability — Burcu

Some interviewers will be HR staff with no technical background. So the system must be easy to use. We have three usability requirements.

First: at least 90 percent of interviewers must finish the main task without help. We will test this with real users and count how many finish alone.

Second: calibration must take less than 45 seconds, so the interview is not delayed. We will measure the time in every session.

Third: users must give the report a SUS score of at least 75. The usual average is 68. We will use the SUS questionnaire after users read a sample report.

Slide 8 · Performance — Mehmet

We have three performance requirements, and we can test each one.

First: the webcam must give at least 720p at 30 frames per second. We tested this on our own cameras. A lower quality adds more error, so 720p is the minimum. At the start of each session, the system will check the camera quality.

Second: video call delay must be under 200 milliseconds. This comes from the ITU standard. We will measure it with WebRTC statistics.

Third: the report must be ready within 60 seconds after the session. We will measure this time on our longest sessions.

Slide 9 · Database / ERD — İrem

Our database has six entities: Interviewer, Session, Candidate, Code Stimulus, Scanpath and Dataset.

An interviewer can run many sessions. A session can show many code snippets. Many candidates can join a session. A scanpath is the order in which one person looks at the code blocks. Scanpaths come from candidates and from the datasets.

We also have some rules. The result can only be novice or experienced. Similarity is between 0 and 1. If there is no consent, there is no data.

We never save the video in the database. We delete it after the analysis.

Slide 10 · Software estimation — Mehmet

To estimate our size, we found similar open-source code for each part and counted the lines. Our total size is about 2,800 lines of code.

Basic COCOMO says we need about 1,080 hours. COCOMO II says about 2,075 hours. Our team capacity is 952 hours. This is four people, for 28 weeks, at 8.5 hours a week.

So COCOMO II is more than two times our capacity. To fix this, we build the most important functions first, F1 to F6. We reuse existing code where we can. And we check our hours at the end of every sprint.

Slide 11 · Sprint progress — Burcu, then Sude

Burcu: In Sprint 1 we finished 36 tasks, about 94 hours of work. We wrote all parts of the SRS. We compared 11 studies and 10 datasets. We chose MiroTalk, and we measured the minimum camera quality.

Some things did not go well. We changed the video platform in the middle of the sprint. We added new tasks late. Most tasks were closed in the last two days. And the work was not shared equally.

In Sprint 2 we will track our time better and keep tasks small. We will also test the eye-tracking models and start building the first paths with the Peitek dataset.

Sude: That is the end of our presentation. Thank you for listening. We are happy to answer your questions.